

■ Scrutor: Comprehensive Project Technical & Architecture Guide

End-to-End Deep Dive into Machine Learning Algorithms, Technology Trade-offs, Data Scaling, and System Architecture

1. Step-by-Step Project Process & System Workflow

The **Scrutor** application was engineered in 6 distinct sequential phases to create a production-grade predictive maintenance platform:

- **Step 1: Telemetry Data Definition & Range Guardrails:** Defined 7 operational sensor features (Voltage, Current, Motor Speed, Motor Temp, Vibration, Ambient Temp, Humidity) and established physical min/max operating bounds to prevent out-of-range data contamination.
- **Step 2: Preprocessing & Dual-Stage Feature Scaling:** Raw sensor inputs are first Min-Max normalized into a [0.0, 1.0] interval, then transformed via Scikit-Learn StandardScaler ($z = (x - \mu) / \sigma$) to standardize feature variance across different physical units.
- **Step 3: Machine Learning Classifier Selection & Training:** Trained a 200-tree Random Forest Classifier on historical industrial machine telemetry datasets to classify machine state into 4 diagnostic classes (Normal, Warning, Worst Condition, Critical) with confidence probability scoring.
- **Step 4: Asynchronous Backend API & In-Memory Preloading:** Built an ASGI microservice using FastAPI. Utilized FastAPI lifespan context to load pickled ML binaries into memory on boot, delivering sub-10ms inference latencies.
- **Step 5: Interactive Client Dashboard & Range Validation:** Engineered a Next.js 15 SPA with React Hook Form and Zod schemas, enforcing real-time validation for blank/zero/out-of-range sensor inputs before dispatches.
- **Step 6: Production Full-Stack Cloud Deployment:** Deployed backend container to Render and frontend Next.js application to Vercel, enabling global accessibility with automatic CORS security rules.

2. Machine Learning Model Trade-offs & Algorithm Comparison

Below is the detailed technical comparison explaining why **Random Forest Classifier** was selected over alternative algorithms:

Algorithm	Pros / Suitability	Cons / Limitations	Verdict for Scrutor
Random Forest (Selected)	Handles non-linear sensor feature interactions; robust to outliers.	Slightly higher overfitting potential; single model inference.	SELECTED: Best probability & explainability for tabular data.
Single Decision Tree	Extremely fast execution and simple rule paths.	High variance; prone to severe overfitting on noisy data.	REJECTED: Poor generalization on noisy industrial telemetry.
Logistic Regression	Mathematically simple and highly interpretable.	Assumes linear decision boundaries; fails on complex non-linear relationships.	REJECTED: Cannot capture complex interactions between thermal and vibration features.
Support Vector Machine (SVM)	Effective in high-dimensional feature spaces.	Slow training and inference; probability calibration is difficult.	REJECTED: Too slow and expensive for real-time confidence scoring.
Deep Neural Network (MLP)	Can model highly complex non-linear functions.	Requires huge training datasets; acts as a black box.	REJECTED: Overkill for initial features; lacks quick iteration.

3. Backend Architecture: Why FastAPI Was Selected

Framework	Architecture Type	Execution Performance	Why Chosen / Rejected
FastAPI (Selected)	ASGI (Asynchronous)	Ultra-high throughput (~20,000 req/sec via uvicorn).	SELECTED: Native async, Pydantic type safety, automatic Swagger docs.
Flask	WSGI (Synchronous)	Moderate throughput; blocking sync execution.	REJECTED: Requires add-on libraries for async & Pydantic; slower uWSGI.
Django	WSGI Monolith	Heavy memory footprint due to ORM & admin.	REJECTED: Over-engineered for microservice ML inference endpoint.

4. Frontend Architecture: Why Next.js 15 & Tailwind CSS Were Selected

Technology	Selected Technology	Alternative Considered	Why Selected Over Alternative
------------	---------------------	------------------------	-------------------------------

Framework	Next.js 15 (App Router)	Vite / Create React App	Next.js provides production build optimizations, file-based routing, and built-in TypeScript support.
Styling	Tailwind CSS v4	Vanilla CSS / Bootstrap	Tailwind utility-first architecture allows rapid dark-mode UI design, border radius, and spacing control.
Form Engine	React Hook Form + Zod	Manual State (`useState`)	React Hook Form prevents re-rendering the entire form on every keystroke. Zod provides runtime validation.

5. Reference Telemetry Values for 4 Diagnostic Conditions

Telemetry Parameter	Valid Range	Normal State	Warning State	Worst Condition	Critical State
Voltage (V)	180 - 260 V	220.0 V	251.1 V	232.0 V	191.6 V
Current (A)	0.1 - 30 A	6.7 A	11.1 A	7.7 A	6.1 A
Motor Speed (RPM)	10 - 3000 RPM	1500 RPM	1311 RPM	1401 RPM	1114 RPM
Motor Temp (°C)	1 - 120 °C	32.2 °C	106.1 °C	29.9 °C	97.7 °C
Vibration (g)	0.01 - 5 g	0.20 g	4.00 g	0.80 g	0.50 g
Ambient Temp (°C)	-10 - 60 °C	25.0 °C	39.0 °C	-1.7 °C	48.2 °C
Humidity (%)	1 - 100 %	35.0 %	10.8 %	65.3 %	10.4 %
Confidence Rating	0 - 100%	100.0%	93.0%	98.5%	99.5%

6. Physics & Engineering Explanations of Diagnostic States

- 1. Normal State (Class 0):** Core temperature (32.2°C) and vibration (0.20g) are low, indicating minimal mechanical friction. Current draw (6.7A) confirms baseline electrical power consumption.
- 2. Warning State (Class 1):** High vibration (4.00g) combined with elevated motor temperature (106.1°C) indicates early bearing wear or shaft misalignment. Maintenance required.
- 3. Worst Condition State (Class 2):** Sub-zero ambient temperature (-1.7°C) and high humidity (65.3%) thicken gearbox lubricant, causing viscous friction and motor speed reduction (1401 RPM).
- 4. Critical State (Class 3):** Low supply voltage (191.6V) reduces torque, causing rotation speed collapse (1114 RPM) while high ambient heat (48.2°C) causes severe thermal overload. Demands emergency shutdown.